Installation CodeIgniter

1. Installation de CodeIgniter

1.1 Télécharger CodeIgniter

1.2 Dézipper dans wamp64/www

WAMP ne peut exécuter que les fichiers qui se trouvent dans le dossier 'www'. C'est comme un espace de travail où ton serveur local cherche les projets web à exécuter.

1.3 Renommer le dossier (exemple monProjetWeb)

Le nom par défaut est générique et compliqué. En le renommant, tu facilites l'accès à ton projet via l'URL (ex: localhost/monProjetWeb) et tu organises mieux tes différents projets.

2. Configuration initiale

2.1. Activer l'exension intl qui se trouve dans php.ini (;extension=intl → extension=intl)

L'extension 'intl' permet à PHP de gérer l'internationalisation (dates, monnaies, langues différentes). CodeIgniter 4 en a BESOIN pour fonctionner. Le ';' au début désactive la ligne, en l'enlevant tu l'actives.

2.2. Copier index.php et .htaccess depuis le dossier public/ vers la racine du projet et modifier le chemin dans index.php ('../app/Config/Paths.php'; → 'app/Config/Paths.php';)

Par défaut, CodeIgniter attend que tu accèdes via /public/. En copiant ces fichiers à la racine, tu peux accéder directement à localhost/tonProjet au lieu de localhost/tonProjet/public. C'est plus propre et professionnel.

Le '../' signifie 'remonte d'un dossier'. Comme tu as déplacé index.php à la racine, il ne doit plus remonter pour trouver le dossier 'app'. Tu corriges donc le chemin pour qu'il trouve les bons fichiers.

2.3. Tester l'installation (http://localhost/monProjetWeb)

3. Activer le mode développement

3.1 Renommer le fichier env (env → .env)

Le fichier '.env' contient les configurations d'environnement (développement/production). Le point au début le rend 'caché' et indique à CodeIgniter que c'est un fichier de configuration actif.

3.2 Editer le fichier .env (#CI_ENVIRONMENT = production → CI_ENVIRONMENT = development) ou créer une nouvelle ligne juste en dessous CI_ENVIRONMENT = development

En mode 'development', CodeIgniter affiche TOUTES les erreurs en détail (super pour apprendre et déboguer). En mode 'production', il cache les erreurs pour la sécurité.

4. Personnalise la page

4.1. Modifier welcome_message.php

welcome_message.php est la page d'accueil par défaut. En la modifiant, tu vérifies que tes changements s'affichent bien et tu comprends où se trouvent les vues (Views = ce que l'utilisateur voit).

4.2. Vérifier les changements

5. Créer une nouvelle page (mvc)

5.1. Ajouter une fonction dans Home.php :

```
public function details()
{
   return view('details');
}
```

Le CONTRÔLEUR est le chef d'orchestre. La fonction details() dit : 'quand on m'appelle, j'affiche la vue details'. C'est la logique de ton application. Une fonction = une action possible.

5.2. Créer la vue details.php en creant un fichier details.php avec ceci par exemple :

```
<!DOCTYPE html>
<html>
<body>
   <h1>Page de détails</h1>
   <p>Voici ma page de détails !</p>
</body>
</html>
```

La VUE (View) est ce que l'utilisateur voit à l'écran. C'est du HTML pur. Elle ne contient QUE l'affichage, pas de logique. Séparation des responsabilités : le contrôleur décide, la vue affiche.

5.3. Ajouter une route dans le fichier app/Config/Routes.php il faut ajouter :

```
$routes->get('details', 'Home::details');
```

Le ROUTING crée des URLs personnalisées et propres. Au lieu de /Home/details (qui montre la structure interne), tu peux avoir juste /details (plus joli et professionnel). Tu définis tes propres chemins !

5.4. Ajouter un lien a welcome_message.php :

```
<a href="/VotreProjet/details">Voir les détails</a>
```

Tu crées une navigation entre tes pages.

5.5. Tester le lien avec l'adresse suivante :
http://localhost/VotreProjet/Home/details
Tu vérifies que tout le cycle MVC fonctionne : clic sur lien → routing trouve la route → appelle le contrôleur Home → exécute details() → affiche la vue details.php. C'est le cœur de CodeIgniter !

6. Passer des données à la vue

6.1. Modifier la fonction details() dans Home.php en rajoutant ceci :

```
public function details()
{
   $data['monMessage'] = "Salut depuis le contrôleur !";
   return view('details', $data);
}
```

Le contrôleur prépare les DONNÉES pour la vue. Ici, tu crées un message dans le contrôleur et tu l'envoies à la vue. Séparation : le contrôleur calcule/prépare, la vue affiche. $data est comme un paquet qu'on envoie.

6.2. Afficher dans la vue en ajoutant dans details.php cette ligne : <?php echo $monMessage; ?>
La vue REÇOIT les données du contrôleur. Grâce à $data['monMessage'] envoyé par le contrôleur, la vue peut utiliser $monMessage directement. C'est la communication Contrôleur → Vue. La vue ne crée pas les données, elle les affiche.

7. Creer une base de données

7.1. Creer la base de données avec phpMyAdmin

La BASE DE DONNÉES stocke les informations de façon permanente (utilisateurs, articles, etc.). Sans elle, tout disparaît quand tu fermes le site. C'est la mémoire de ton application !

7.2. Crer la table users avec ce type de colonnes :

id (INT, AUTO_INCREMENT, PRIMARY KEY)
name (VARCHAR 100)
email (VARCHAR 100)

Une TABLE organise les données en lignes et colonnes (comme Excel). 'id' identifie chaque utilisateur de façon unique (AUTO_INCREMENT = numéro automatique). 'name' et 'email' stockent les infos. PRIMARY KEY = l'identifiant principal unique.

7.3. Configurer Database.php et modifier ceci :

```
'username' => 'root',
'password' => '',
'database' => 'nomDeVotreProjet',
```

Tu donnes à CodeIgniter les IDENTIFIANTS pour se connecter à la base de données. C'est comme donner un nom d'utilisateur et mot de passe pour accéder à un coffre-fort. Sans ça, CodeIgniter ne peut pas lire/écrire dans la BD.

7.4. Créer le modele UserModel.php dans app/Models et y ajouter ceci :

```
<?php namespace App\Models;
use CodeIgniter\Model;

class UserModel extends Model
{
   protected $table = 'users';
   protected $primaryKey = 'id';
   protected $returnType = 'array';
   protected $allowedFields = ['name', 'email'];
}
```

Le MODÈLE (Model) communique avec la base de données. Il représente la table 'users' en code PHP. $table = nom de la table, $allowedFields = colonnes qu'on peut modifier. C'est l'intermédiaire entre ton code et la BD. Tu ne fais jamais de SQL direct, le modèle s'en charge !

8. Afficher les utilisateurs

8.1. Créer la fonction allUsers() dans Home.php avec ceci :

```
public function allUsers()
{
   $userModel = new \App\Models\UserModel();
   $users = $userModel->findAll();
   $data['users'] = $users;
   return view('allUsers', $data);
}
```

Le contrôleur ORCHESTRE : 1) Il crée un UserModel pour parler à la BD, 2) findAll() demande TOUS les utilisateurs, 3) Il met les résultats dans $data, 4) Il envoie tout à la vue. C'est le cycle complet : Contrôleur → Modèle → Base de données → Modèle → Contrôleur → Vue

8.2. Créer la vue allUsers.php dans app/Vieuws et y inserer ceci :

```
<!DOCTYPE html>
<html>
<body>
   <h1>Liste des utilisateurs</h1>
   <?php foreach($users as $user): ?>
      <p><?= $user['name'] ?> - <?= $user['email'] ?></p>
   <?php endforeach; ?>
</body>
</html>
```

$users est un TABLEAU d'utilisateurs envoyé par le contrôleur. foreach() parcourt chaque utilisateur un par un. Pour chaque $user, on affiche son nom et email. C'est comme lire une liste et afficher chaque élément. La vue transforme les données en HTML visible.

8.3. Ajouter le lien dans welcome_message.php :

```
<a href="/VotreProjet/allUsers">Voir tous les utilisateurs</a>
```

Tu crées la NAVIGATION vers ta nouvelle page. Quand on clique, CodeIgniter appelle Home::allUsers() qui récupère les données et affiche la liste. C'est le cycle complet d'une application web dynamique !

8.4. Inserer la route dans Routes.php :

```
$routes->get('allUsers', 'Home::allUsers');
```